Table of Contents

1 Exploratory Data Analysis and Deep Learning

1.1 Breast Cancer Diagnosis Using the Wisconsin Diagnostic Dataset

Student Name: _____
Register / Roll No.: _____
Department / Year: _____
Course / Subject: _____
Date of Submission: _____

1.2 Abstract

This report presents an exploratory data analysis and deep-learning case study using the Wisconsin Breast Cancer Diagnostic dataset. The dataset contains 569 cases, 30 continuous numerical features describing cell-nucleus characteristics, and a benign/malignant diagnosis. Data quality was checked for missing values, duplicate rows and extreme values; visualizations were used to study diagnosis balance, feature distributions, correlations, multivariate structure and class differences. A standardized multilayer perceptron (30-64-32-1) was trained with Adam and binary cross-entropy, using a stratified 70/15/15 train/validation/test split and validation-loss early stopping. On one fixed 86-case test split, the MLP achieved 1.000 accuracy and ROC-AUC, while Logistic Regression and Random Forest each achieved 0.988 accuracy. These results demonstrate a complete coursework pipeline but are not evidence of clinical readiness; broader validation is essential.

1.3 1. Introduction and Objectives

Exploratory data analysis (EDA) helps identify data quality issues, class balance, feature distributions, unusual values and relationships among variables before and alongside modelling. This case study uses numerical features computed from digitized fine-needle aspirate images of breast masses. The model operates on 30 tabular measurements, not original image pixels.

Objectives - Describe a real-world dataset with 30 numerical predictors and a binary diagnosis. - Inspect missing values, exact duplicates, outlier flags, distributions and correlated features. - Create a minimum of five explained visuals; this expanded report contains 29 figure outputs. - Standardize data without fitting the scaler on held-out partitions. - Train a

PyTorch multilayer perceptron and evaluate it using multiple metrics and charts. - Compare the MLP with Logistic Regression and Random Forest baselines. - Discuss reproducibility, limitations, and future improvements.

1.4 2. Dataset Description

Dataset source: UCI Machine Learning Repository — Breast Cancer Wisconsin (Diagnostic). Scikit-learn also distributes a mirror of this dataset. The dataset has 569 instances and 30 continuous predictors. The target has 357 benign cases (62.7%) and 212 malignant cases (37.3%). The feature families are radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension, each represented as a mean, standard error, and worst value.

Property	Value
Observations	569
Numerical predictors	30
Benign cases	357 (62.7%)
Malignant cases	212 (37.3%)
Missing cells	0
Exact duplicate rows	0
Task	Binary classification

Full descriptive statistics are exported as `outputs/feature_summary.csv`. Extreme values are flagged with the 1.5×IQR rule but are not automatically deleted, since statistical extremeness does not prove a measurement is invalid.

1.5 3. EDA and Preprocessing

The script checks dataset shape, missing values, exact duplicate rows and label frequencies. The target is encoded explicitly as `y=1` for malignant and `y=0` for benign. A stratified split produces 398 training, 85 validation, and 86 test cases. The class counts are 250 benign / 148 malignant in training, 53 / 32 in validation, and 54 / 32 in testing.

`StandardScaler` is fitted only on the training partition, then applied unchanged to validation and test data. This helps prevent data leakage. IQR-based outlier counts are saved to `outputs/iqr_outlier_counts.csv`. The 30 predictors include strongly correlated size-related measurements, so interpretation of individual variables should be cautious.

1.6 4. Deep Learning Model

The model architecture is **30 inputs → Dense(64) + ReLU + Dropout(0.20) → Dense(32) + ReLU + Dropout(0.10) → one output**

logit. It has 4,097 trainable parameters. The sigmoid converts the output logit into a malignant-class probability during evaluation.

Parameter	Setting
Loss	BCEWithLogitsLoss
Optimizer	Adam
Learning rate	0.001
Weight decay	0.0001
Batch size	32
Dropout	0.20 and 0.10
Maximum epochs	150
Early-stopping patience	15 epochs
Random seed	42

The best validation-loss checkpoint is restored before final test evaluation. The test set is not used for early stopping.

1.7 5. Results and Evaluation

On the fixed 86-case test split, the MLP achieved accuracy 1.0000, malignant precision 1.0000, malignant recall 1.0000, malignant F1 1.0000, ROC-AUC 1.0000 and average precision 1.0000. Its confusion matrix contains 54 true negatives, 32 true positives, zero false positives and zero false negatives.

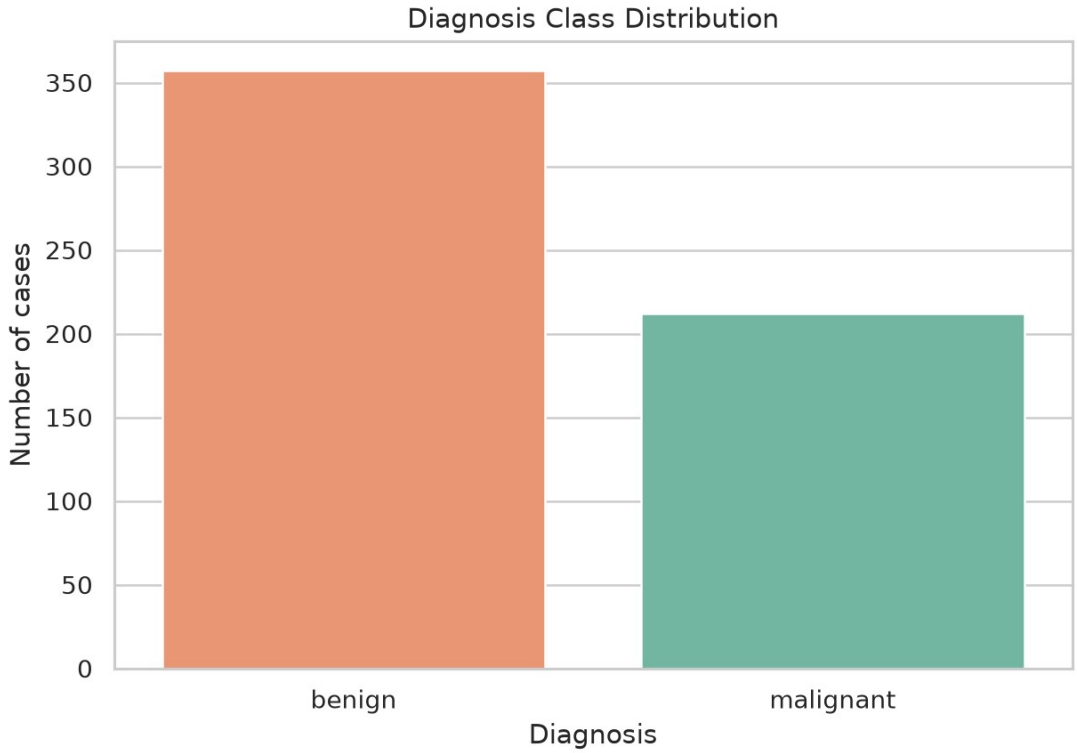
Model	Accuracy	Malignant precision	Malignant recall	Malignant F1	ROC-AUC
MLP	1.0000	1.0000	1.0000	1.0000	1.0000
Logistic Regression	0.9884	0.9697	1.0000	0.9846	0.9988
Random Forest	0.9884	1.0000	0.9688	0.9841	1.0000

These are split-specific results, not proof of generalization. The test set includes only 32 malignant cases, so confidence in sensitivity is limited by sample size. Repeated stratified cross-validation and external validation are recommended.

1.8 6. Visualization Gallery

The following charts are generated by `breast_cancer_eda_mlp.py` and saved under `outputs/`. Each caption explains the visual and its limitations.

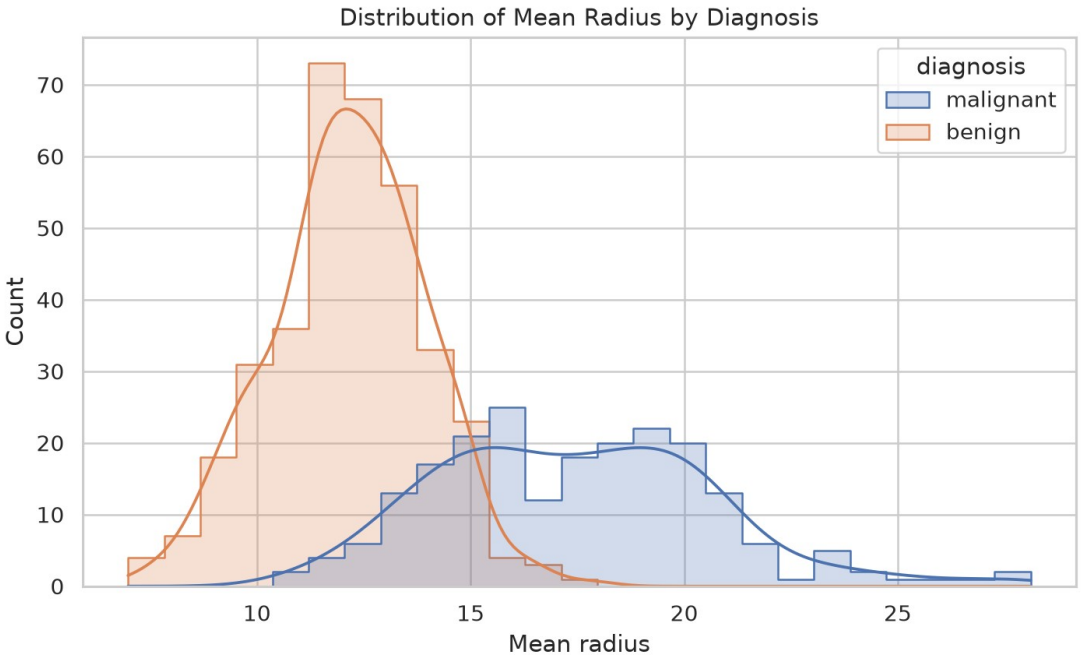
1.8.1 Figure 1. Diagnosis class distribution



Diagnosis class distribution

There are 357 benign and 212 malignant cases. Stratification is used to preserve this ratio in the train, validation and test partitions.

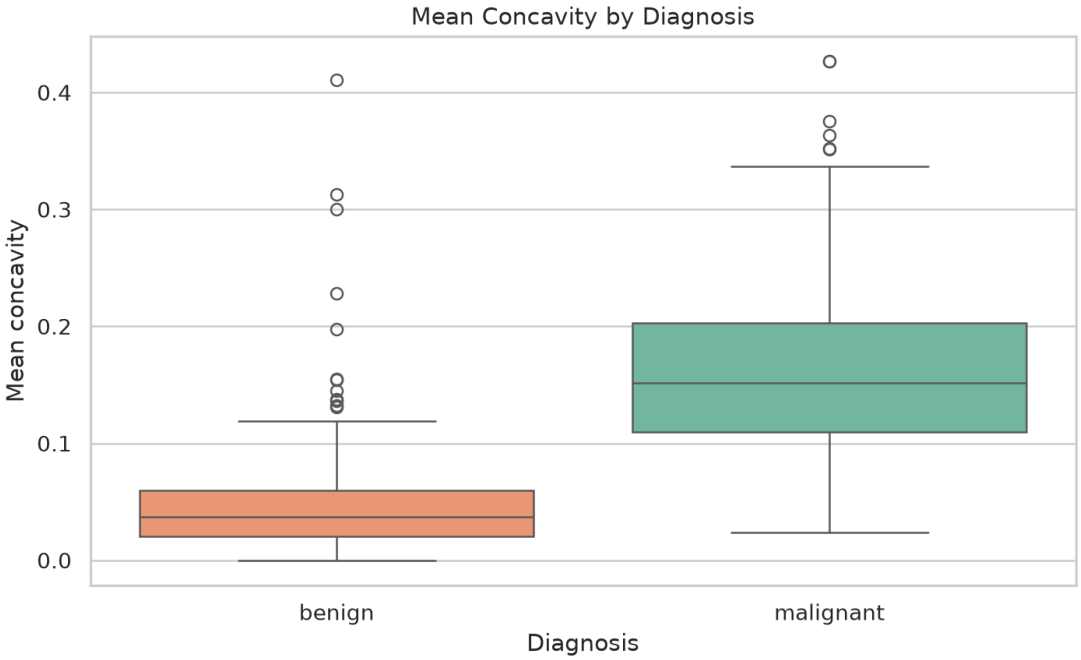
1.8.2 Figure 2. Mean radius by diagnosis



Mean radius by diagnosis

Malignant cases tend to have larger mean radius, although the distributions overlap. A single measurement is not a diagnostic rule.

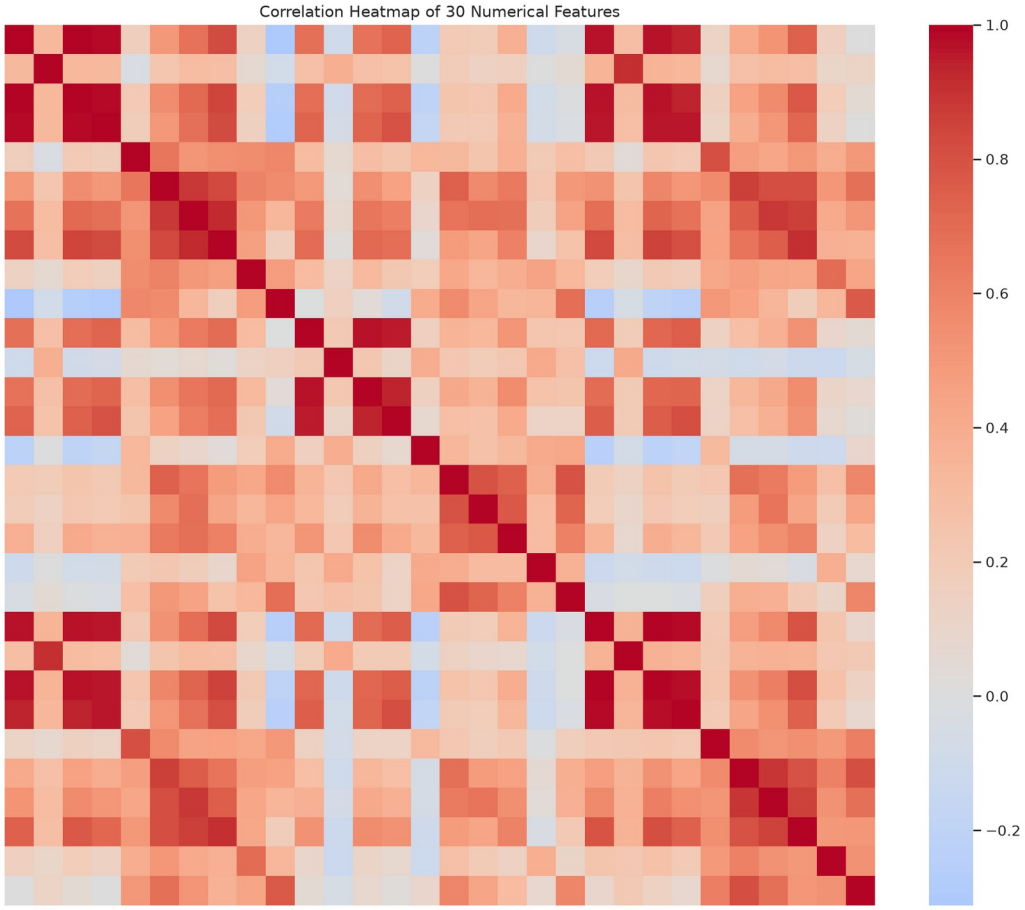
1.8.3 Figure 3. Mean concavity by diagnosis



Mean concavity by diagnosis

The malignant group has a higher median mean concavity in this sample, supporting a group-level difference but not causation.

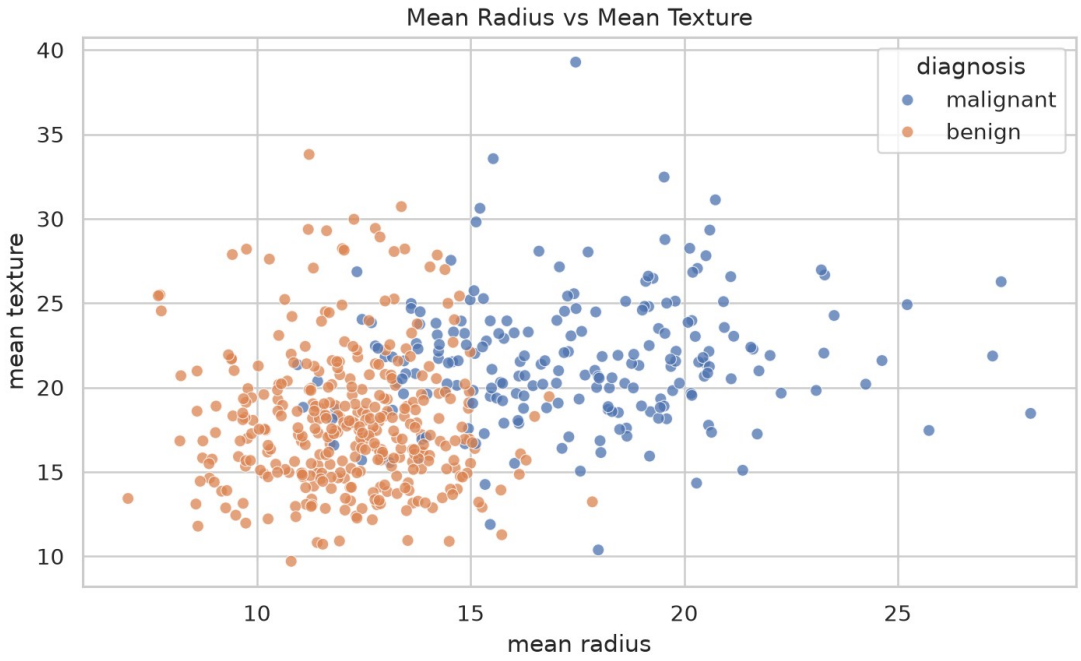
1.8.4 Figure 4. Correlation heatmap of all 30 predictors



Correlation heatmap of all 30 predictors

Size-related variables are strongly correlated, so some predictors may carry overlapping information.

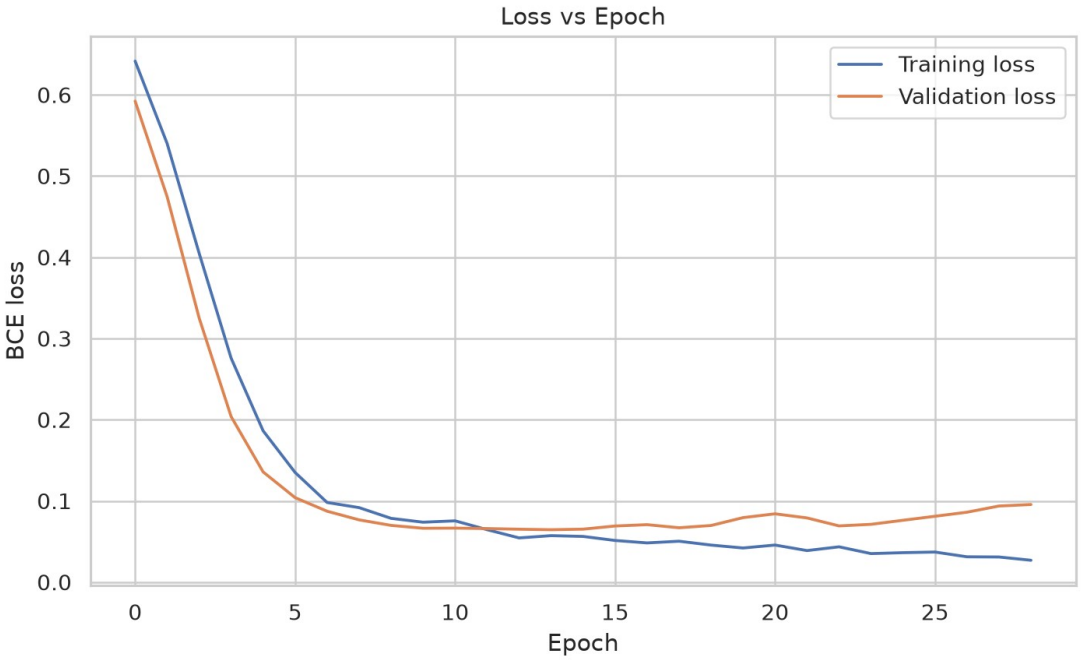
1.8.5 Figure 5. Mean radius versus mean texture



Mean radius versus mean texture

The class groups occupy partly different regions but overlap in two dimensions.

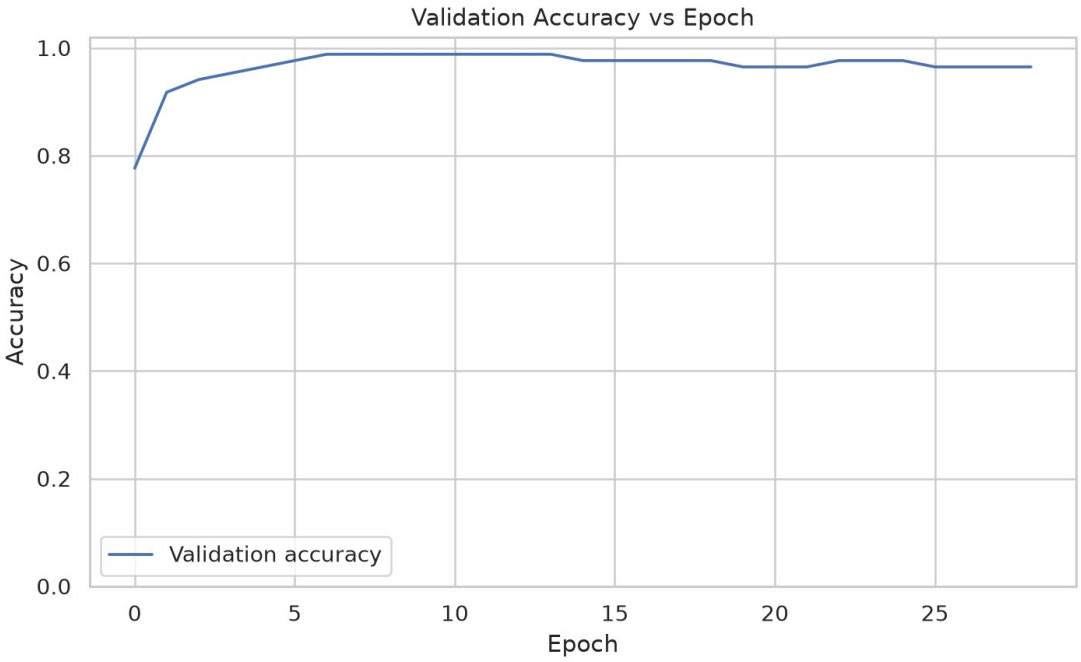
1.8.6 Figure 6. Training and validation loss



Training and validation loss

Loss decreases rapidly early in training. Early stopping restores the checkpoint associated with the best validation loss.

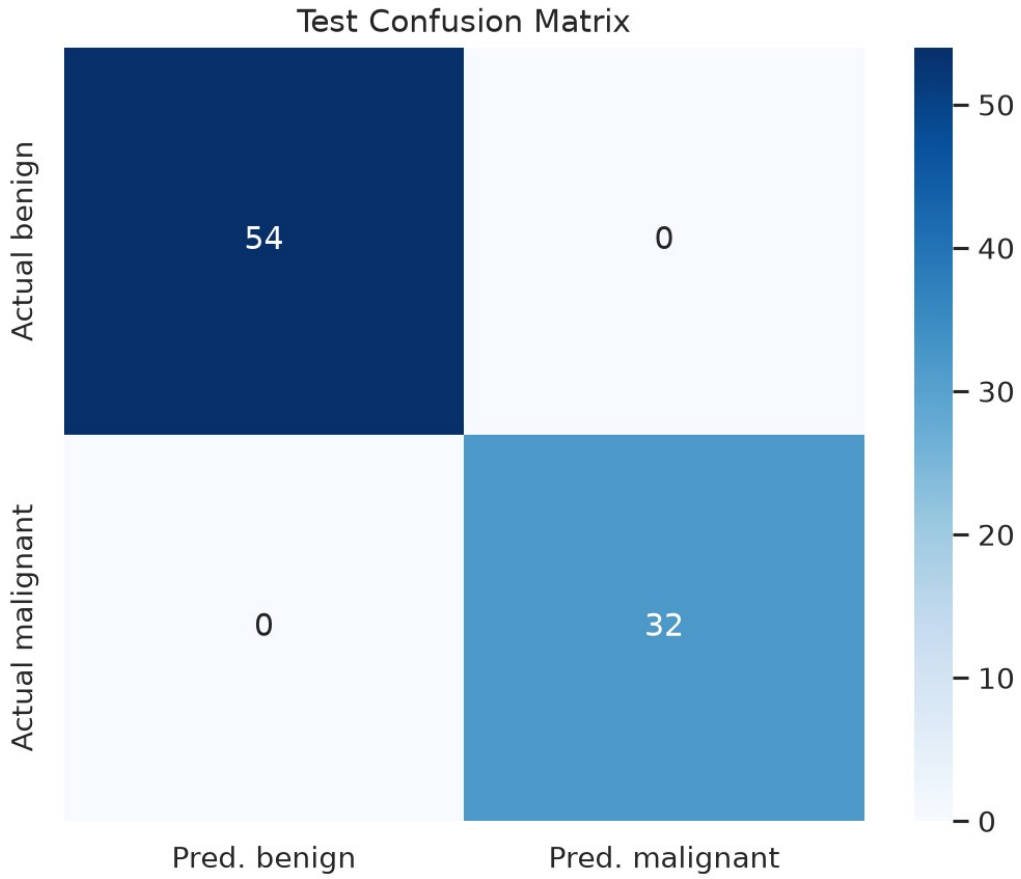
1.8.7 Figure 7. Validation accuracy by epoch



Validation accuracy by epoch

Validation accuracy rises rapidly, then fluctuates slightly. The validation set contains only 85 cases.

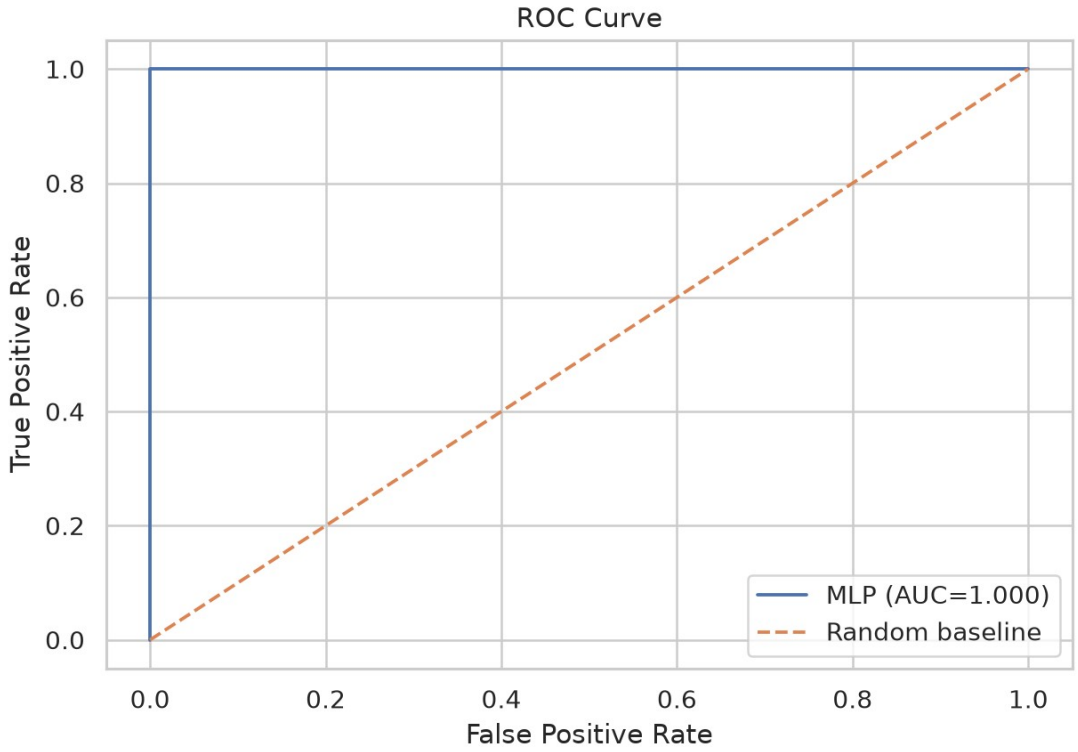
1.8.8 Figure 8. Held-out test confusion matrix



Held-out test confusion matrix

All 54 benign and 32 malignant test cases were classified correctly in this split; small-sample uncertainty remains.

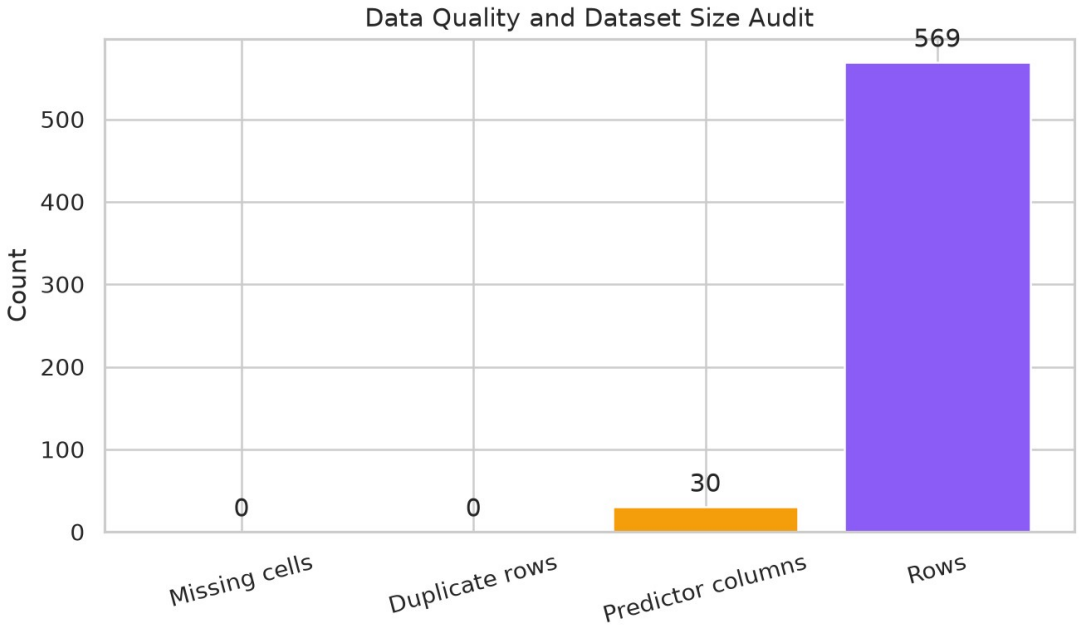
1.8.9 Figure 9. ROC curve and AUC



ROC curve and AUC

AUC is 1.000 on this test partition only. ROC-AUC is not a calibration measure.

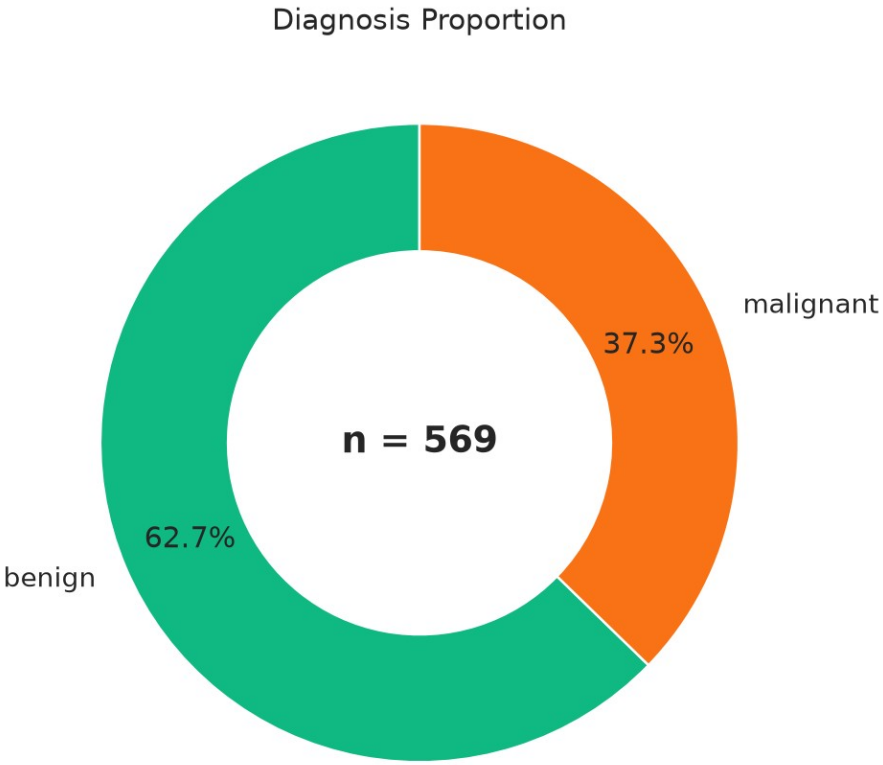
1.8.10 Figure 10. Data quality and dataset size audit



Data quality and dataset size audit

The loaded table has 569 rows, 30 predictors, no missing cells and no exact duplicate rows under the performed checks.

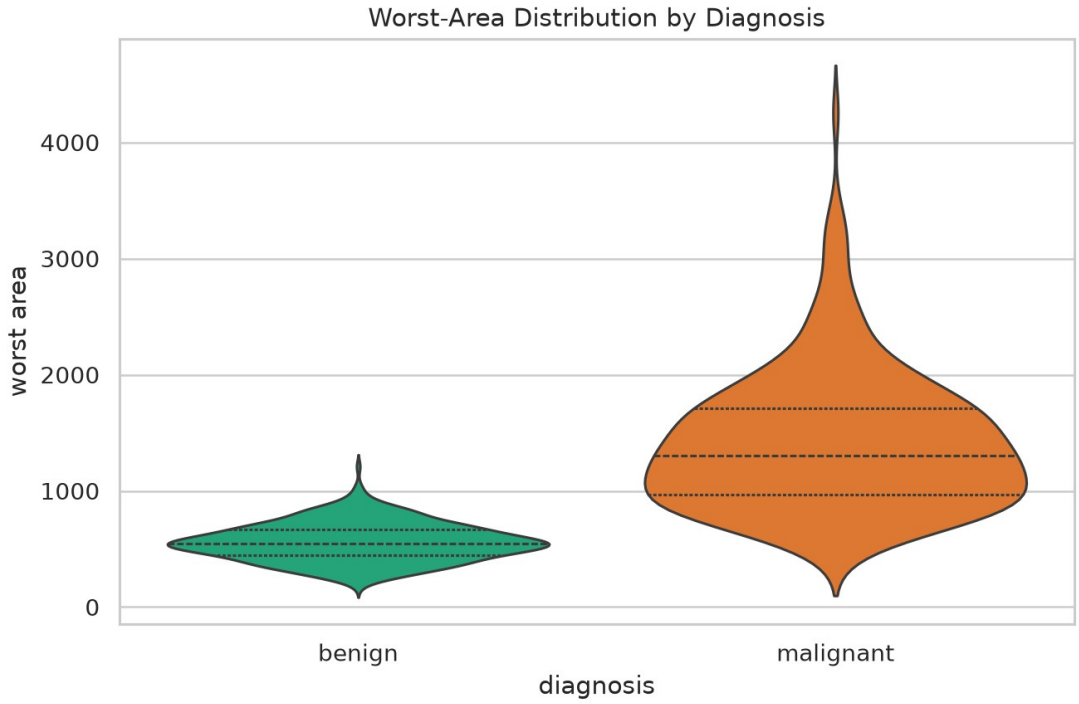
1.8.11 Figure 11. Diagnosis proportion



Diagnosis proportion

The chart expresses the target ratio as percentages: 62.7% benign and 37.3% malignant.

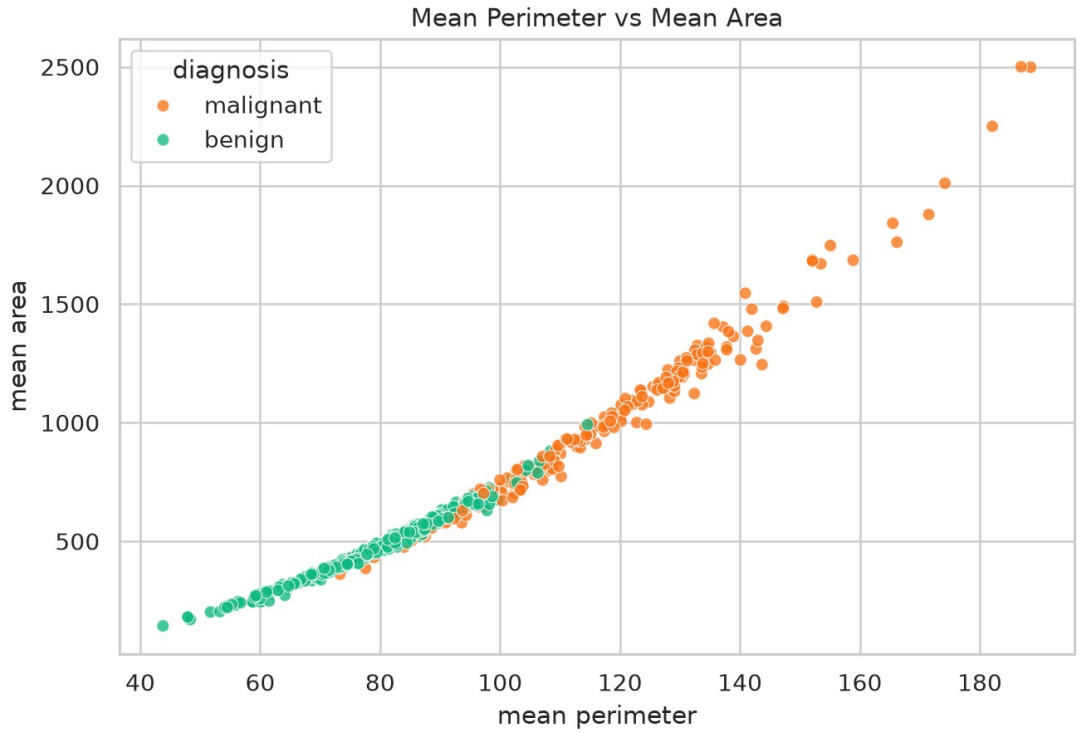
1.8.12 Figure 12. Worst-area distribution by diagnosis



Worst-area distribution by diagnosis

The malignant group shows larger worst-area values and a higher-value tail. Extreme values are not automatically errors.

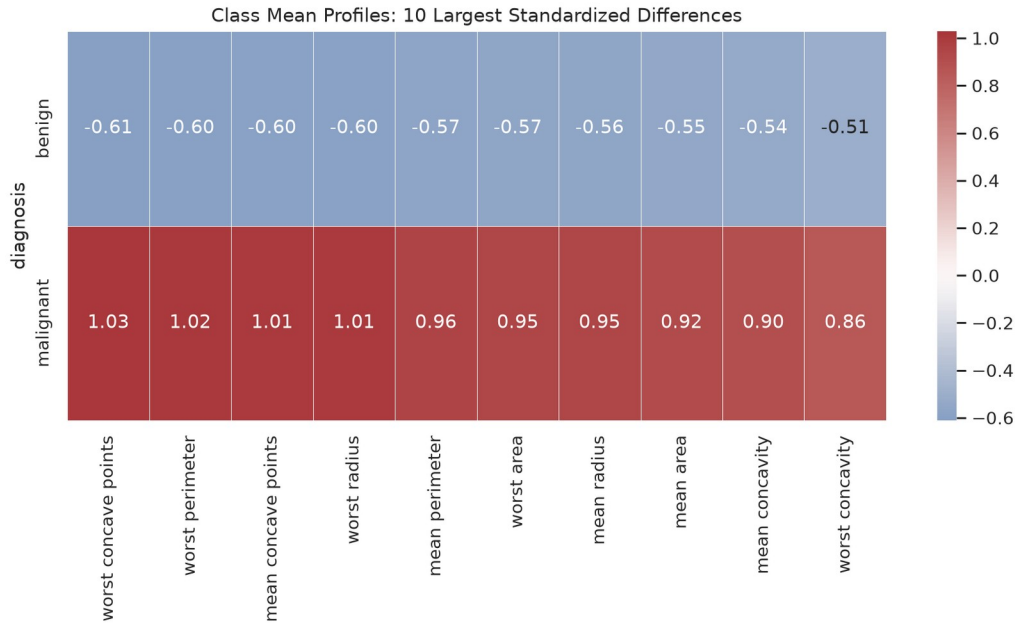
1.8.13 Figure 13. Mean perimeter versus mean area



Mean perimeter versus mean area

The strong positive relationship reflects related geometric characteristics of nucleus size.

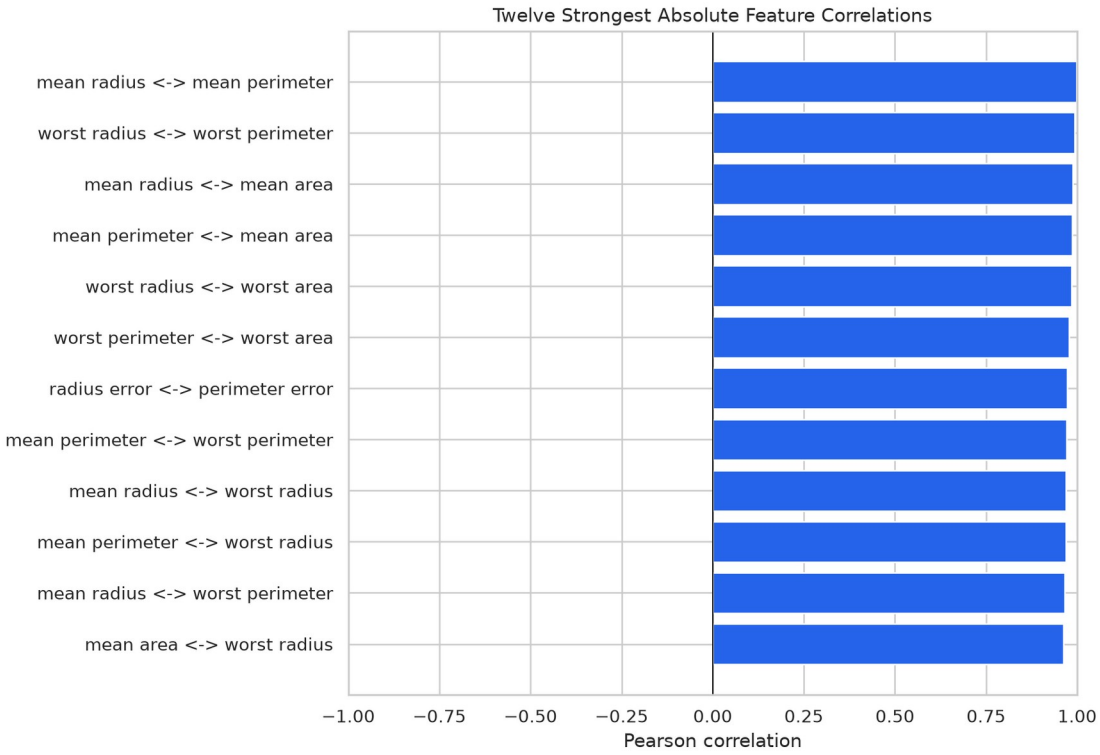
1.8.14 Figure 14. Standardized class mean profiles



Standardized class mean profiles

Ten features with the largest standardized class-mean differences are displayed on a common scale for visual comparison.

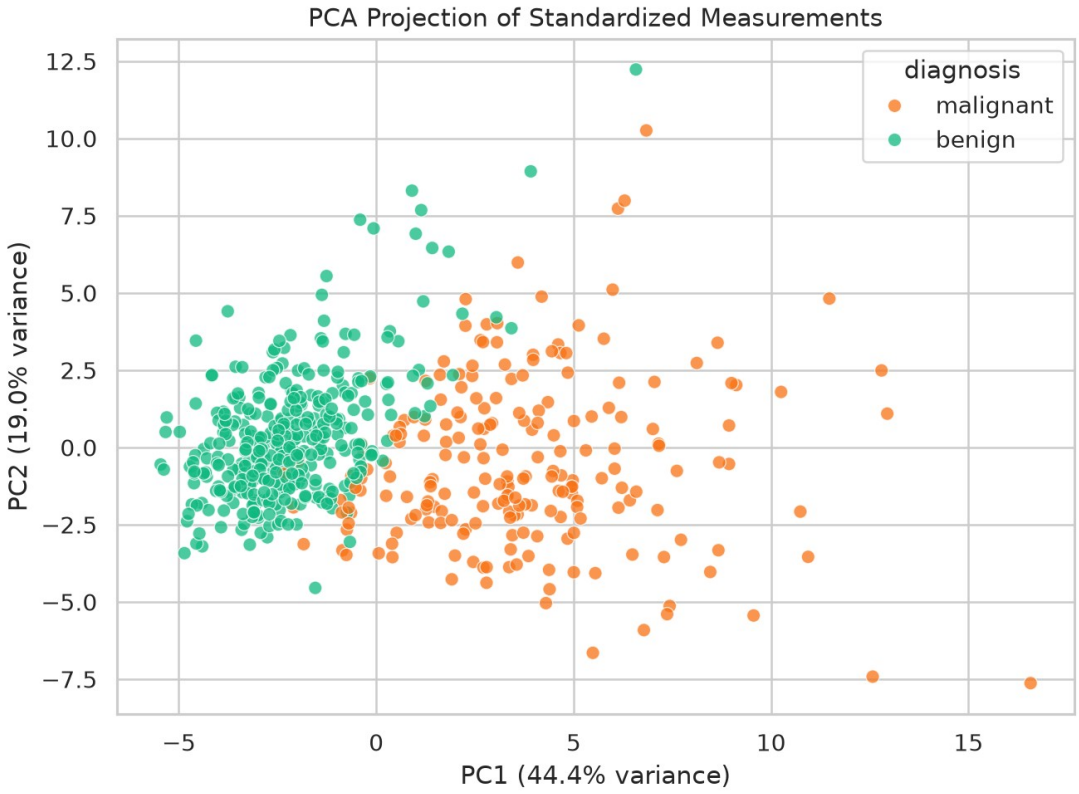
1.8.15 Figure 15. Twelve strongest absolute correlations



Twelve strongest absolute correlations

The strongest pairs are largely size-related measurements. Correlation does not imply causation.

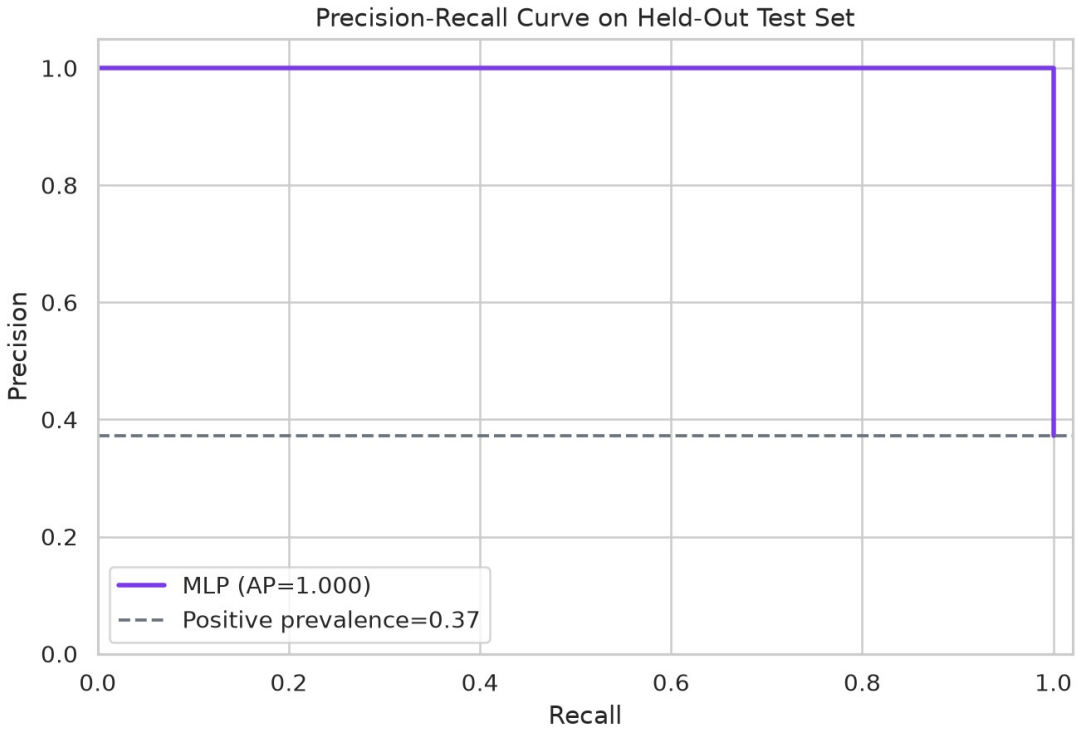
1.8.16 Figure 16. PCA projection of standardized features



PCA projection of standardized features

The first two principal components show partial separation and overlap; only two components are displayed.

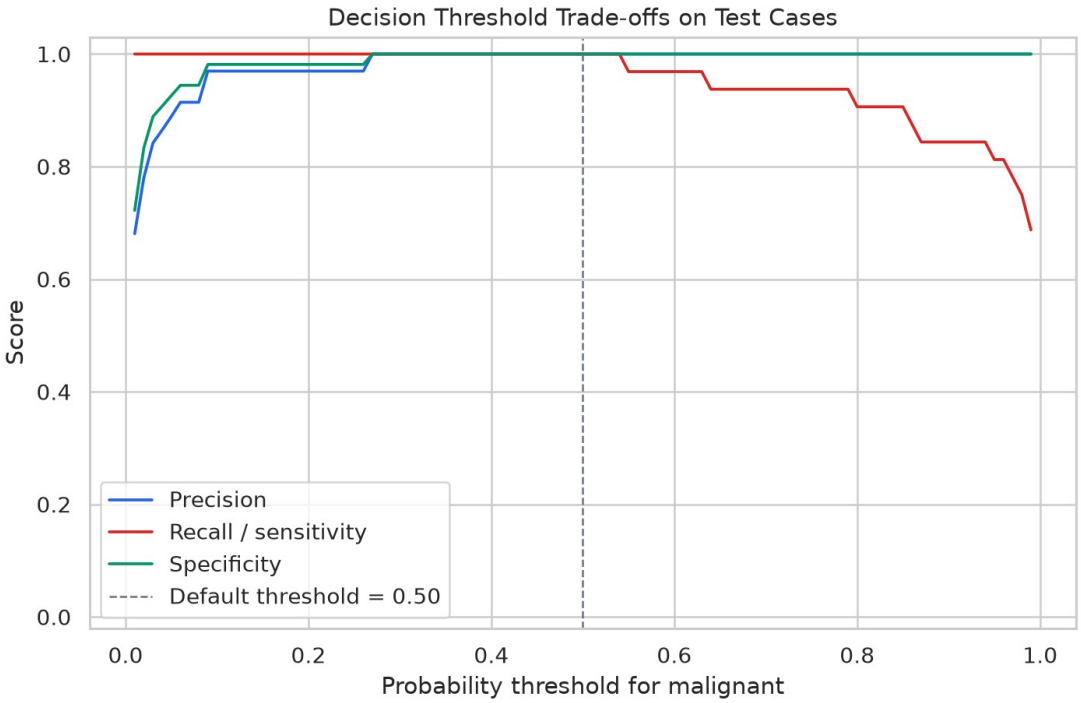
1.8.17 Figure 17. Precision-recall curve



Precision-recall curve

Average precision is 1.000 on this test split. Precision-recall analysis focuses on the malignant positive class.

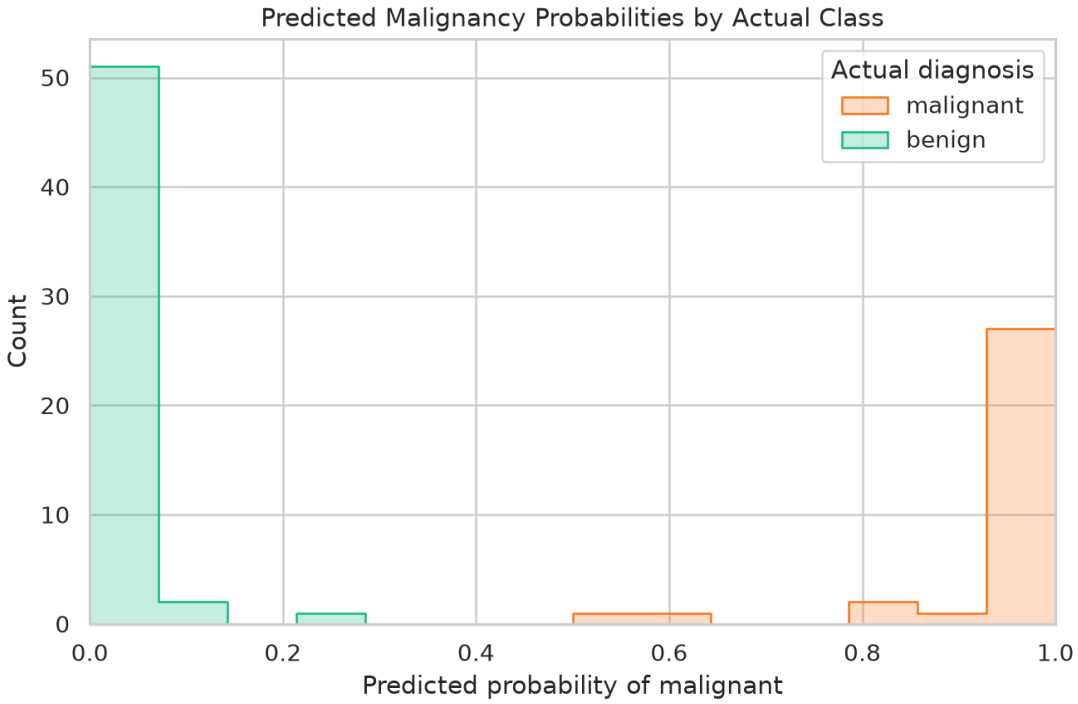
1.8.18 Figure 18. Probability threshold trade-offs



Probability threshold trade-offs

Precision, sensitivity and specificity change with the threshold. A 0.50 threshold is a software default, not a clinically justified setting.

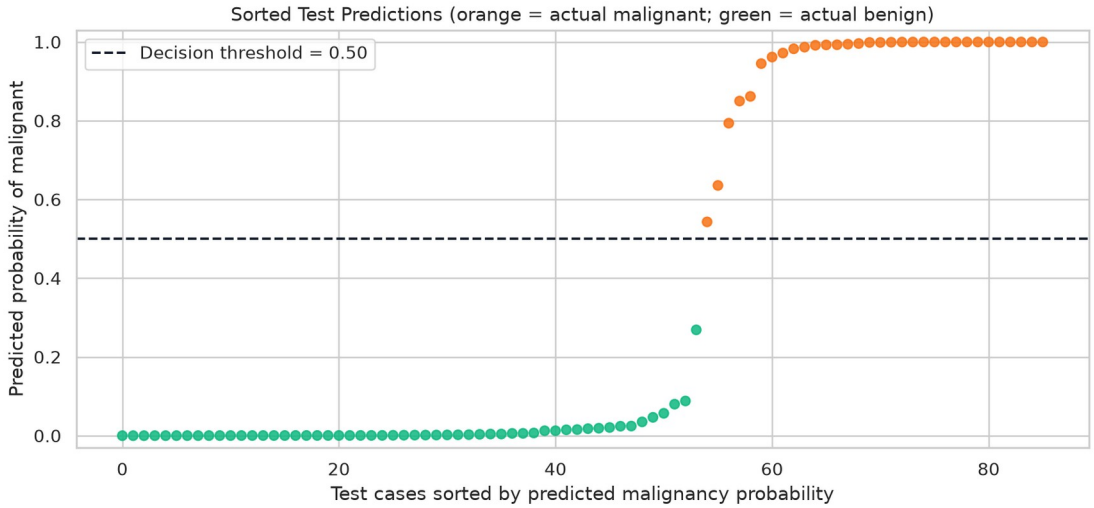
1.8.19 Figure 19. Predicted probabilities by actual class



Predicted probabilities by actual class

Test probabilities are well separated by actual label in this run, but external data is needed to assess generalization.

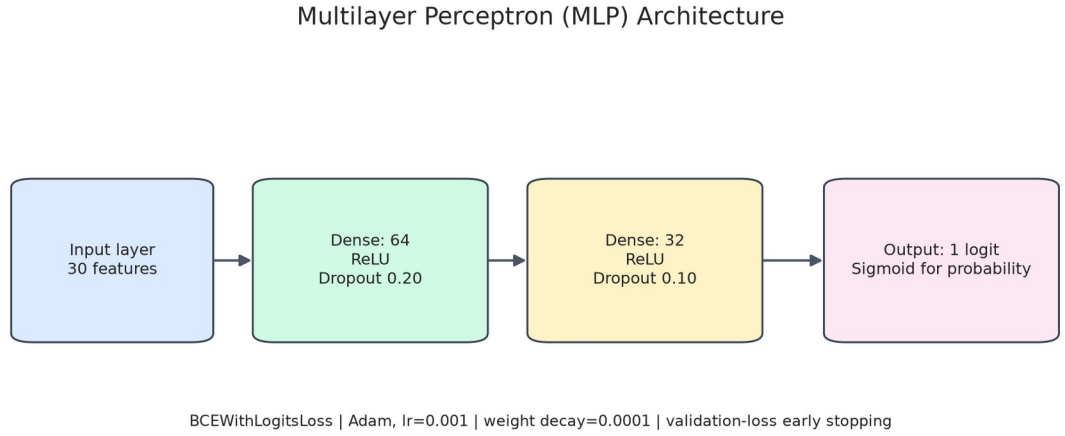
1.8.20 Figure 20. Sorted test predictions



Sorted test predictions

Every test case is on the correct side of the 0.50 threshold in this split; this visual reflects only 86 test cases.

1.8.21 Figure 21. MLP architecture



MLP architecture

The network has 30 inputs, hidden layers of 64 and 32 units, dropout and one output logit.

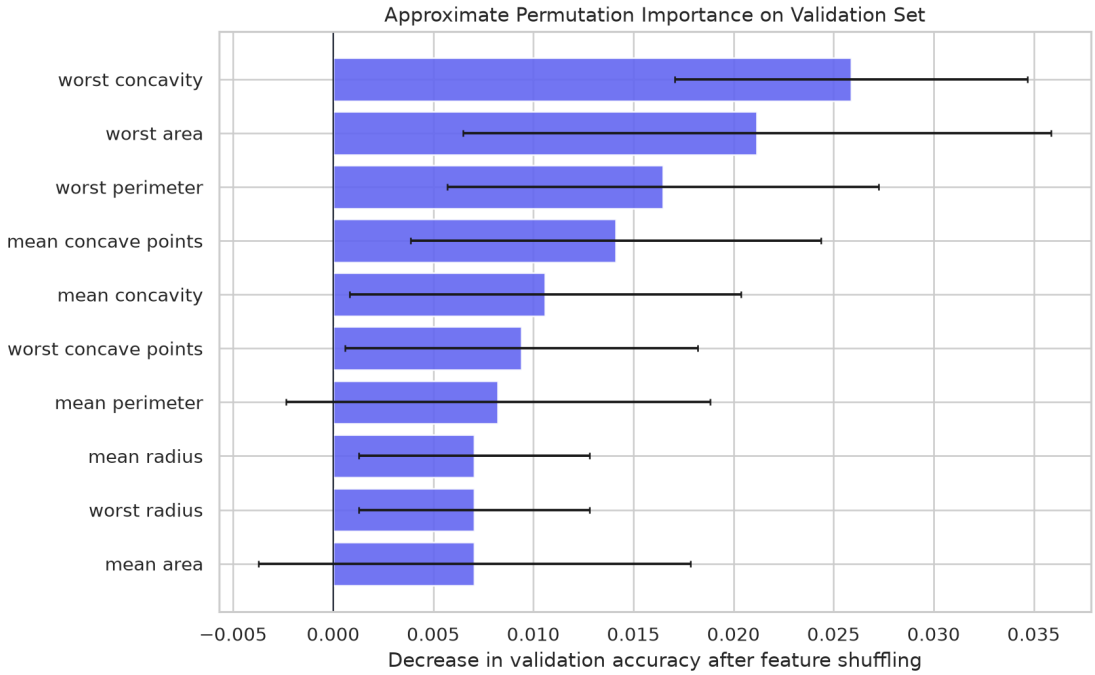
1.8.22 Figure 22. Class counts across data splits



Class counts across data splits

Stratification preserves approximately the same ratio across 398 train, 85 validation and 86 test observations.

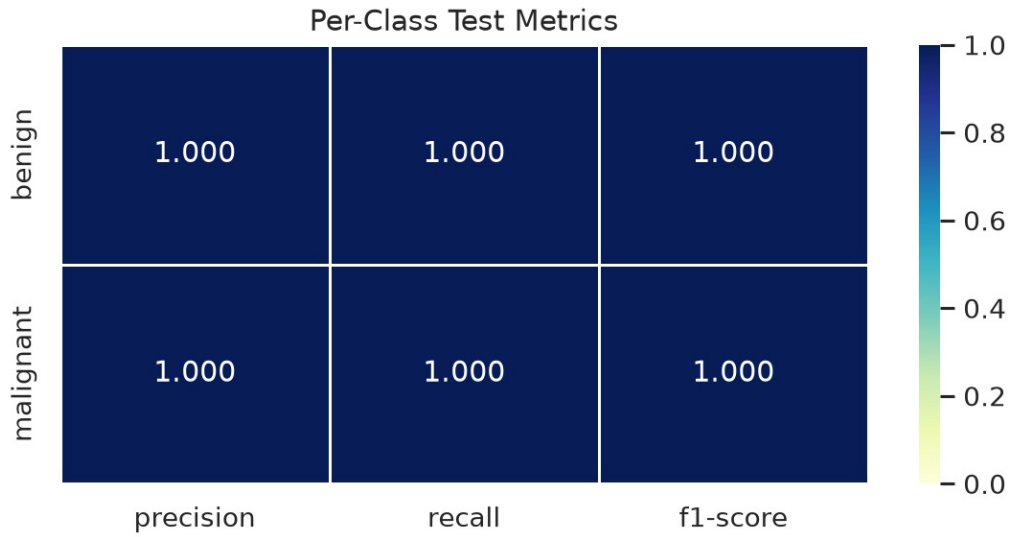
1.8.23 Figure 23. Approximate permutation importance



Approximate permutation importance

Validation accuracy drops are small and variable for correlated features; treat these scores as exploratory, not definitive.

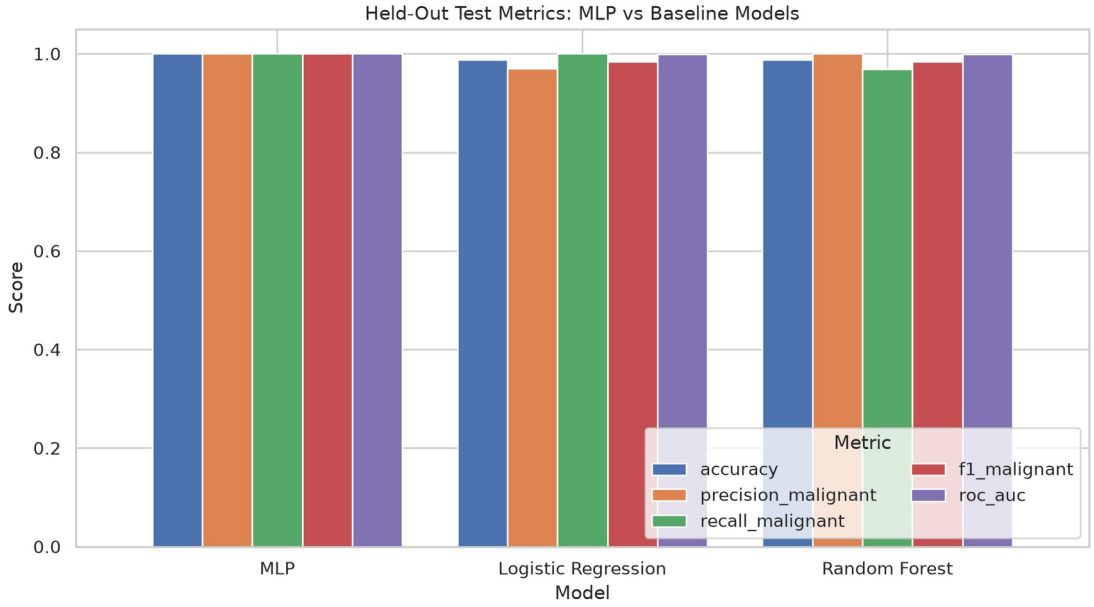
1.8.24 Figure 24. Per-class precision, recall and F1



Per-class precision, recall and F1

Both classes score 1.000 on this split; this does not remove uncertainty from the small hold-out set.

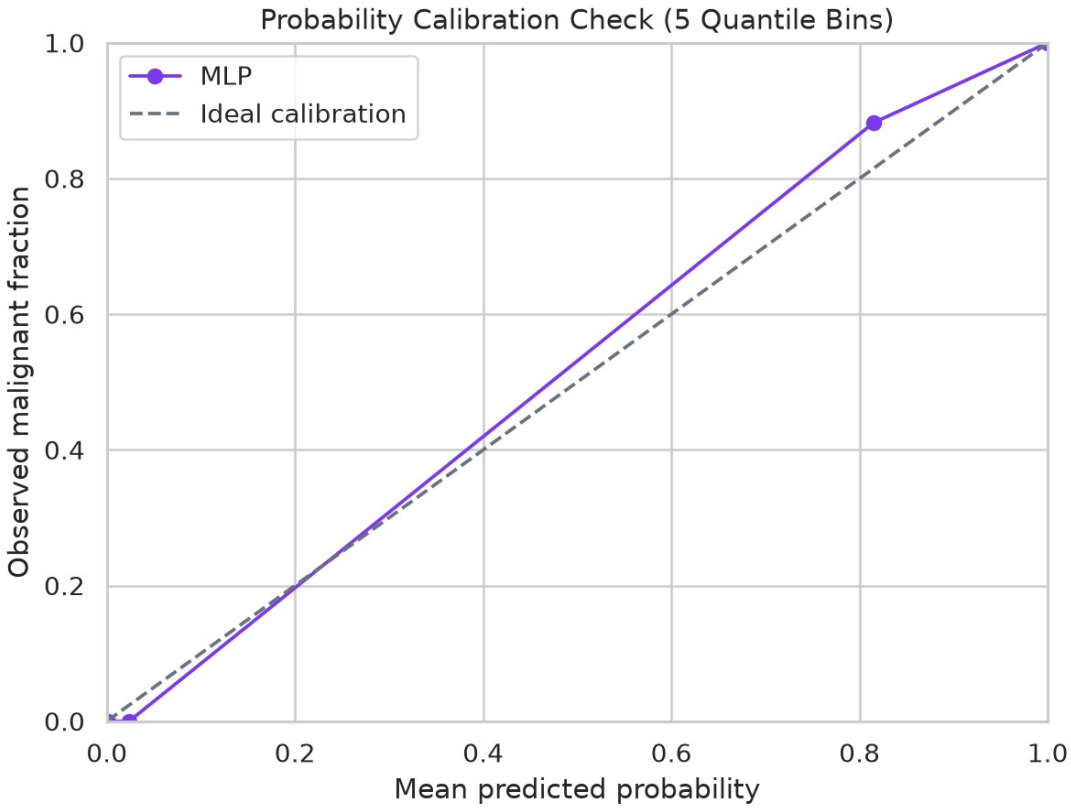
1.8.25 Figure 25. MLP versus conventional baselines



MLP versus conventional baselines

The MLP scored 1.000 accuracy while the two baselines scored 0.988; one split is insufficient to claim general superiority.

1.8.26 Figure 26. Probability calibration check



Probability calibration check

The binned curve is coarse because the test set is small. High AUC does not guarantee well-calibrated probabilities.

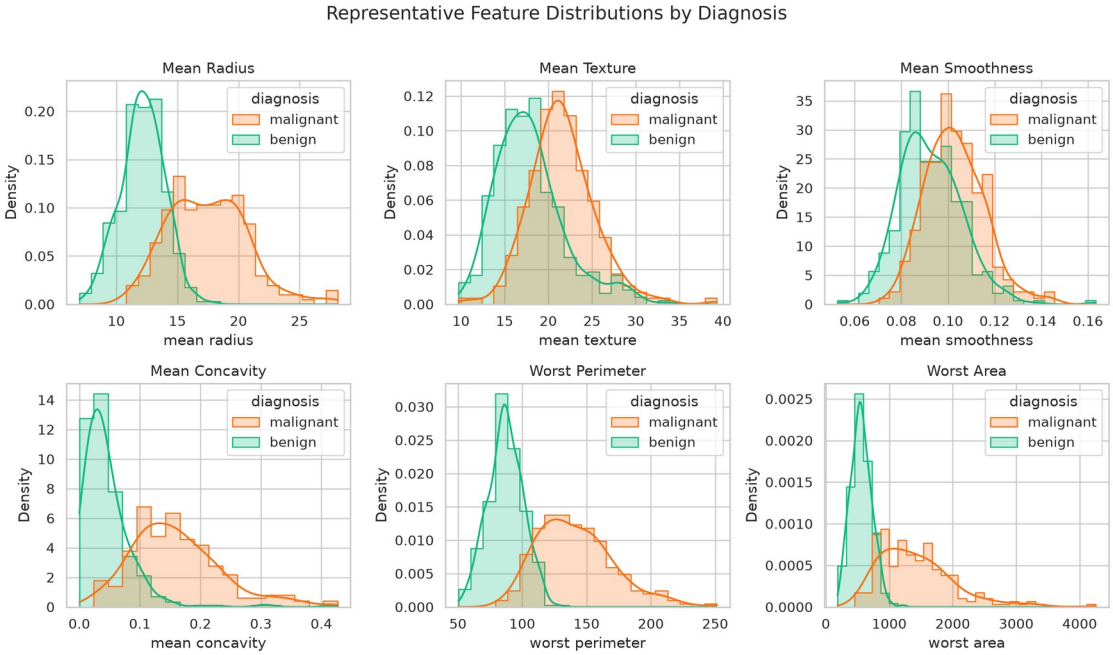
1.8.27 Figure 27. Pairwise relationships among selected features



Pairwise relationships among selected features

The pairplot shows distribution shapes and pairwise correlations among four representative features.

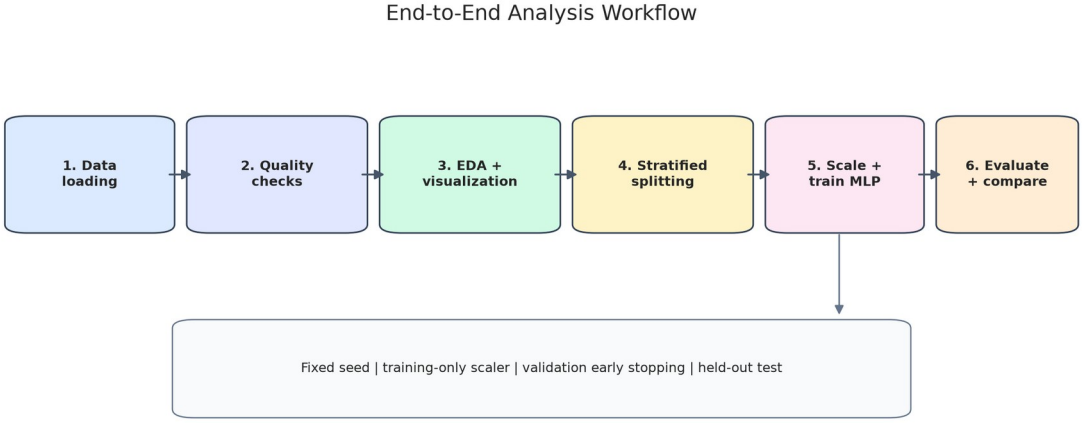
1.8.28 Figure 28. Representative feature distribution grid



Representative feature distribution grid

Six distributions cover size, texture, smoothness and concavity, showing differing degrees of group overlap.

1.8.29 Figure 29. End-to-end analysis workflow



End-to-end analysis workflow

The diagram summarizes data checks, EDA, stratification, scaling, MLP training and final test evaluation.

1.9 7. Discussion and Limitations

- **Single split:** performance may change under another train/test assignment. Repeated stratified cross-validation is recommended.

- **Small test set:** 86 cases, including 32 malignant cases, provide limited statistical confidence.
- **Population shift:** model behaviour may change if features are measured under different processes or in different populations.
- **Correlated predictors:** feature-attribution scores may be unstable because related features can substitute for one another.
- **Calibration and threshold:** a strong ROC-AUC does not guarantee accurate risk probabilities. A decision threshold requires context-specific assessment.
- **Clinical safety:** this is a classroom demonstration, not a medical device, screening tool or diagnostic service.

1.10 8. Conclusion and Future Scope

The case study demonstrates data loading, descriptive analysis, 29 figures, a PyTorch MLP, baseline comparisons and multi-metric evaluation. The MLP performed best on the fixed split, while the conventional baselines also performed strongly. Future work should include repeated cross-validation, external validation, calibration analysis, confidence intervals, subgroup performance checks, threshold analysis and expert review.

1.11 9. References

1. UCI Machine Learning Repository. “Breast Cancer Wisconsin (Diagnostic).” https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic
2. scikit-learn developers. “load_breast_cancer” documentation. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html
3. Street, W. N., Wolberg, W. H., & Mangasarian, O. L. (1993). Nuclear feature extraction for breast tumor diagnosis. *Biomedical Image Processing and Biomedical Visualization*, SPIE, 861–870.
4. Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874. https://doi.org/10.1016/j.patrec.2005.10.010
5. Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.
6. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
7. Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3), e0118432. https://doi.org/10.1371/journal.pone.0118432

1.12 Appendix A. Run Instructions

```
python -m venv .venv
# Windows PowerShell: .venv\\Scripts\\Activate.ps1
# macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
python breast_cancer_eda_mlp.py
```

The complete commented source file is `breast_cancer_eda_mlp.py`. The run generates chart images, training history, feature summaries, outlier counts, baseline metrics, permutation-importance results and `metrics.json`.

Disclaimer: This project is educational only and must not be used for medical decisions.

1.13 Appendix B. Complete Python Source Code

```
"""
Breast Cancer Wisconsin Diagnostic: EDA + MLP
Run: python breast_cancer_eda_mlp.py
Dependencies: numpy, pandas, matplotlib, seaborn, scikit-learn, torch
"""
from pathlib import Path
import json, random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (accuracy_score, precision_score,
recall_score, f1_score,
                             confusion_matrix, classification_report,
roc_curve, roc_auc_score,
                             precision_recall_curve,
average_precision_score)
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.calibration import calibration_curve
import torch
from torch import nn
from torch.utils.data import TensorDataset, DataLoader

OUT = Path("outputs")
OUT.mkdir(exist_ok=True)
SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)

# 1. Load data (scikit-learn mirror of the UCI Wisconsin Diagnostic
dataset)
```

```
raw = load_breast_cancer(as_frame=True)
df = raw.frame.copy()
# Dataset target is 0=malignant, 1=benign in sklearn; map to
clinically clear labels.
df["diagnosis"] = df["target"].map({0: "malignant", 1: "benign"})
df.drop(columns=["target"], inplace=True)
feature_cols = list(raw.feature_names)
X = df[feature_cols].to_numpy(dtype=np.float32)
# Positive class for ROC is malignant.
y = (df["diagnosis"] == "malignant").astype(np.int64).to_numpy()

print("Dataset shape:", df.shape)
print("Missing cells:", int(df.isna().sum().sum()))
print("Duplicate rows:", int(df.duplicated().sum()))
print(df["diagnosis"].value_counts())

# 2. EDA and saved visualizations
sns.set_theme(style="whitegrid")
plt.figure(figsize=(7,5))
sns.countplot(data=df, x="diagnosis", hue="diagnosis",
order=["benign","malignant"], palette="Set2", legend=False)
plt.title("Diagnosis Class Distribution"); plt.xlabel("Diagnosis");
plt.ylabel("Number of cases")
plt.tight_layout(); plt.savefig(OUT/"01_class_distribution.png",
dpi=180); plt.close()

plt.figure(figsize=(8,5))
sns.histplot(data=df, x="mean radius", hue="diagnosis", kde=True,
bins=25, element="step")
plt.title("Distribution of Mean Radius by Diagnosis");
plt.xlabel("Mean radius"); plt.tight_layout()
plt.savefig(OUT/"02_mean_radius_distribution.png", dpi=180);
plt.close()

plt.figure(figsize=(8,5))
sns.boxplot(data=df, x="diagnosis", y="mean concavity",
hue="diagnosis", order=["benign","malignant"], palette="Set2",
legend=False)
plt.title("Mean Concavity by Diagnosis"); plt.xlabel("Diagnosis");
plt.ylabel("Mean concavity")
plt.tight_layout(); plt.savefig(OUT/"03_concavity_boxplot.png",
dpi=180); plt.close()

corr = df[feature_cols].corr()
plt.figure(figsize=(12,10))
sns.heatmap(corr, cmap="coolwarm", center=0, square=False,
xticklabels=False, yticklabels=False)
plt.title("Correlation Heatmap of 30 Numerical Features")
plt.tight_layout(); plt.savefig(OUT/"04_correlation_heatmap.png",
dpi=180); plt.close()
```

```
plt.figure(figsize=(8,5))
sns.scatterplot(data=df, x="mean radius", y="mean texture",
hue="diagnosis", alpha=.75)
plt.title("Mean Radius vs Mean Texture"); plt.tight_layout()
plt.savefig(OUT/"05_radius_texture_scatter.png", dpi=180); plt.close()

# Distribution summaries, feature correlations, and simple outlier
counts (IQR rule)
df[feature_cols].describe().T.to_csv(OUT/"feature_summary.csv")
q1=df[feature_cols].quantile(.25); q3=df[feature_cols].quantile(.75);
iqr=q3-q1
outlier_counts=((df[feature_cols] < (q1-1.5*iqr)) | (df[feature_cols]
> (q3+1.5*iqr))).sum()
outlier_counts.to_csv(OUT/"iqr_outlier_counts.csv",
header=["IQR_outlier_count"])

# 3. Split train / validation / test (70/15/15); stratify to preserve
class ratio.
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=.30, random_state=SEED, stratify=y)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=.50, random_state=SEED, stratify=y_temp)

# Fit scaler only on training data to avoid data leakage.
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train).astype(np.float32)
X_val=scaler.transform(X_val).astype(np.float32)
X_test=scaler.transform(X_test).astype(np.float32)

device=torch.device("cuda" if torch.cuda.is_available() else "cpu")
model=nn.Sequential(nn.Linear(30,64), nn.ReLU(), nn.Dropout(.20),
                    nn.Linear(64,32), nn.ReLU(), nn.Dropout(.10),
                    nn.Linear(32,1))
model.to(device)
criterion=nn.BCEWithLogitsLoss()
optimizer=torch.optim.Adam(model.parameters(), lr=.001,
weight_decay=1e-4)
train_ds=TensorDataset(torch.tensor(X_train),torch.tensor(y_train,dtyp
e=torch.float32))
loader=DataLoader(train_ds,batch_size=32,shuffle=True)
Xt=torch.tensor(X_val,device=device);
yt=torch.tensor(y_val,dtype=torch.float32,device=device)
history={"train_loss":[],"val_loss":[],"val_accuracy":[]}
best_val=float("inf"); best_state=None; patience=15; stale=0;
max_epochs=150

for epoch in range(max_epochs):
    model.train(); running=0.0; count=0
    for xb,yb in loader:
```

```
        xb=xb.to(device); yb=yb.to(device).view(-1,1)
        optimizer.zero_grad()
        logits=model(xb); loss=criterion(logits,yb)
        loss.backward(); optimizer.step()
        running += loss.item()*len(xb); count += len(xb)
    train_loss=running/count
    model.eval()
    with torch.no_grad():
        val_logits=model(Xt).view(-1)
        val_loss=criterion(val_logits,yt).item()
        val_pred=(torch.sigmoid(val_logits)>=.5).cpu().numpy()
        val_acc=accuracy_score(y_val,val_pred)
    history["train_loss"].append(train_loss)
    history["val_loss"].append(val_loss)
    history["val_accuracy"].append(val_acc)
    if val_loss < best_val:
        best_val=val_loss; best_state={k:v.detach().cpu().clone() for
k,v in model.state_dict().items()}; stale=0
    else:
        stale+=1
    if stale>=patience: break

model.load_state_dict(best_state); model.eval()
with torch.no_grad():
    logits=model(torch.tensor(X_test,device=device)).view(-
1).cpu().numpy()
probs=1/(1+np.exp(-logits))
pred=(probs>=.5).astype(int)
metrics={
 "test_accuracy":float(accuracy_score(y_test,pred)),

"test_precision_malignant":float(precision_score(y_test,pred,zero_divi
sion=0)),

"test_recall_malignant":float(recall_score(y_test,pred,zero_division=0
)),
 "test_f1_malignant":float(f1_score(y_test,pred,zero_division=0)),
 "test_roc_auc":float(roc_auc_score(y_test,probs)),
 "epochs_run":len(history["train_loss"]),

"train_size":len(y_train),"validation_size":len(y_val),"test_size":len
(y_test),
 "dataset_rows":len(df),"feature_count":len(feature_cols),
}
(OUT/"metrics.json").write_text(json.dumps(metrics,indent=2))
pd.DataFrame(history).to_csv(OUT/"training_history.csv",index_label="e
poch")

# Model performance visualizations
plt.figure(figsize=(8,5))
```

```
plt.plot(history["train_loss"],label="Training loss")
plt.plot(history["val_loss"],label="Validation loss")
plt.title("Loss vs Epoch"); plt.xlabel("Epoch"); plt.ylabel("BCE
loss"); plt.legend(); plt.tight_layout()
plt.savefig(OUT/"06_loss_vs_epoch.png",dpi=180); plt.close()

plt.figure(figsize=(8,5))
plt.plot(history["val_accuracy"],label="Validation accuracy")
plt.title("Validation Accuracy vs Epoch"); plt.xlabel("Epoch");
plt.ylabel("Accuracy"); plt.ylim(0,1.02)
plt.legend(); plt.tight_layout();
plt.savefig(OUT/"07_accuracy_vs_epoch.png",dpi=180); plt.close()

cm=confusion_matrix(y_test,pred,labels=[0,1])
plt.figure(figsize=(6,5))
sns.heatmap(cm,annot=True,fmt="d",cmap="Blues",xticklabels=["Pred.
benign","Pred. malignant"],
            yticklabels=["Actual benign","Actual malignant"])
plt.title("Test Confusion Matrix"); plt.tight_layout();
plt.savefig(OUT/"08_confusion_matrix.png",dpi=180); plt.close()

fpr,tpr,_=roc_curve(y_test,probs)
plt.figure(figsize=(7,5))
plt.plot(fpr,tpr,label=f"MLP (AUC={metrics['test_roc_auc']:.3f})")
plt.plot([0,1],[0,1],"--",label="Random baseline")
plt.xlabel("False Positive Rate"); plt.ylabel("True Positive Rate");
plt.title("ROC Curve")
plt.legend(); plt.tight_layout();
plt.savefig(OUT/"09_roc_curve.png",dpi=180); plt.close()

print("\nTEST METRICS")
print(json.dumps(metrics,indent=2))
print("\nClassification report (0=benign, 1=malignant):")
print(classification_report(y_test,pred,target_names=["benign","malign
ant"],zero_division=0))
print("\nFigures and metrics saved under outputs/")

# Expanded report visuals and baseline comparisons (figures 10-29)
# Baselines are fit to the same training split and evaluated against
the same held-out test set.
benchmarks = {
    "MLP": {
        "accuracy": accuracy_score(y_test, pred),
        "precision_malignant": precision_score(y_test, pred,
zero_division=0),
        "recall_malignant": recall_score(y_test, pred,
zero_division=0),
        "f1_malignant": f1_score(y_test, pred, zero_division=0),
        "roc_auc": roc_auc_score(y_test, probs),
    }
```

```
}
log_reg = LogisticRegression(max_iter=2000, class_weight="balanced",
random_state=SEED)
log_reg.fit(X_train, y_train)
log_probs = log_reg.predict_proba(X_test)[:, 1]
log_pred = (log_probs >= .5).astype(int)
benchmarks["Logistic Regression"] = {
    "accuracy": accuracy_score(y_test, log_pred),
    "precision_malignant": precision_score(y_test, log_pred,
zero_division=0),
    "recall_malignant": recall_score(y_test, log_pred,
zero_division=0),
    "f1_malignant": f1_score(y_test, log_pred, zero_division=0),
    "roc_auc": roc_auc_score(y_test, log_probs),
}
rf = RandomForestClassifier(n_estimators=300, class_weight="balanced",
random_state=SEED, n_jobs=-1)
rf.fit(X_train, y_train)
rf_probs = rf.predict_proba(X_test)[:, 1]
rf_pred = (rf_probs >= .5).astype(int)
benchmarks["Random Forest"] = {
    "accuracy": accuracy_score(y_test, rf_pred),
    "precision_malignant": precision_score(y_test, rf_pred,
zero_division=0),
    "recall_malignant": recall_score(y_test, rf_pred,
zero_division=0),
    "f1_malignant": f1_score(y_test, rf_pred, zero_division=0),
    "roc_auc": roc_auc_score(y_test, rf_probs),
}
bench_df = pd.DataFrame(benchmarks).T
bench_df.index.name = "model"
bench_df.to_csv(OUT / "baseline_model_comparison.csv")

# Data-quality and class-share visuals.
fig, ax = plt.subplots(figsize=(7.5, 4.5))
quality = pd.Series({"Missing cells": int(df.isna().sum().sum()),
"Duplicate rows": int(df.duplicated().sum()),
                     "Predictor columns": len(feature_cols), "Rows":
len(df)})
ax.bar(quality.index, quality.values, color=["#3B82F6", "#10B981",
"#F59E0B", "#8B5CF6"])
ax.set_title("Data Quality and Dataset Size Audit");
ax.set_ylabel("Count"); ax.tick_params(axis="x", rotation=15)
for i, value in enumerate(quality.values): ax.text(i, value +
max(quality.values)*.02, str(value), ha="center", va="bottom")
fig.tight_layout(); fig.savefig(OUT / "10_data_quality_audit.png",
dpi=180); plt.close(fig)

counts = df["diagnosis"].value_counts().reindex(["benign",
"malignant"])
```

```
fig, ax = plt.subplots(figsize=(6.5, 5.2))
ax.pie(counts.values, labels=counts.index, autopct="%1.1f%%",
startangle=90,
       colors=["#10B981", "#F97316"], wedgeprops={"width": .40,
"edgecolor": "white"}, pctdistance=.78)
ax.text(0, 0, f"n = {len(df)}", ha="center", va="center", fontsize=15,
fontweight="bold")
ax.set_title("Diagnosis Proportion"); fig.tight_layout();
fig.savefig(OUT / "11_diagnosis_donut.png", dpi=180); plt.close(fig)

# Additional feature distributions and relationships.
fig, ax = plt.subplots(figsize=(7.5, 5))
sns.violinplot(data=df, x="diagnosis", y="worst area",
hue="diagnosis", order=["benign","malignant"],
               palette={"benign":"#10B981","malignant":"#F97316"},
legend=False, inner="quartile", ax=ax)
ax.set_title("Worst-Area Distribution by Diagnosis");
fig.tight_layout();
fig.savefig(OUT/"12_worst_area_violin.png",dpi=180); plt.close(fig)

fig, ax = plt.subplots(figsize=(7.5,5.2))
sns.scatterplot(data=df,x="mean perimeter",y="mean
area",hue="diagnosis",alpha=.78,

palette={"benign":"#10B981","malignant":"#F97316"},ax=ax)
ax.set_title("Mean Perimeter vs Mean Area"); fig.tight_layout();
fig.savefig(OUT/"13_perimeter_area_scatter.png",dpi=180);
plt.close(fig)

eda_std_df =
pd.DataFrame(StandardScaler().fit_transform(df[feature_cols]),columns=
feature_cols,index=df.index)
eda_std_df["diagnosis"] = df["diagnosis"].values
class_means = eda_std_df.groupby("diagnosis")[feature_cols].mean()
diffs = (class_means.loc["malignant"] -
class_means.loc["benign"]).abs().sort_values(ascending=False)
top_diff_features = diffs.head(10).index.tolist()
fig, ax = plt.subplots(figsize=(10,5.8))
sns.heatmap(class_means[top_diff_features],annot=True,fmt=".2f",cmap="
vlag",center=0,linewidths=.4,ax=ax)
ax.set_title("Class Mean Profiles: 10 Largest Standardized
Differences")
fig.tight_layout();
fig.savefig(OUT/"14_class_mean_profile_heatmap.png",dpi=180);
plt.close(fig)

correlations =
corr.where(np.triu(np.ones(corr.shape),k=1).astype(bool)).stack()
top_corr =
correlations.reindex(correlations.abs().sort_values(ascending=False).h
```

```
ead(12).index).sort_values(key=lambda v:v.abs())
fig, ax = plt.subplots(figsize=(10,7))
ax.barh([f"{a} <-> {b}" for a,b in top_corr.index],top_corr.values,
        color=["#2563EB" if v>=0 else "#DC2626" for v in
top_corr.values])
ax.axvline(0,color="black",lw=.8); ax.set_xlim(-1,1)
ax.set_title("Twelve Strongest Absolute Feature Correlations");
ax.set_xlabel("Pearson correlation")
fig.tight_layout();
fig.savefig(OUT/"15_top_feature_correlations.png",dpi=180);
plt.close(fig)

pca = PCA(n_components=2,random_state=SEED)
coords = pca.fit_transform(scaler.transform(X).astype(np.float32))
pca_df =
pd.DataFrame({"PC1":coords[:,0],"PC2":coords[:,1],"diagnosis":df["diag
nosis"].values})
fig, ax = plt.subplots(figsize=(7.5,5.6))
sns.scatterplot(data=pca_df,x="PC1",y="PC2",hue="diagnosis",alpha=.75,
s=42,

palette={"benign":"#10B981","malignant":"#F97316"},ax=ax)
ax.set_title("PCA Projection of Standardized Measurements")
ax.set_xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}%
variance)")
ax.set_ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}%
variance)")
fig.tight_layout(); fig.savefig(OUT/"16_pca_projection.png",dpi=180);
plt.close(fig)

# Positive class is malignant (label 1): precision-recall and
threshold trade-offs.
pr_precision, pr_recall, _ = precision_recall_curve(y_test,probs)
avg_precision = average_precision_score(y_test,probs)
fig, ax = plt.subplots(figsize=(7.5,5.2))
ax.plot(pr_recall,pr_precision,lw=2.2,color="#7C3AED",label=f"MLP
(AP={avg_precision:.3f})")
ax.axhline(float(y_test.mean()),ls="--",color="#6B7280",label=f"Positi
ve prevalence={y_test.mean():.2f}")
ax.set_xlabel("Recall"); ax.set_ylabel("Precision");
ax.set_title("Precision-Recall Curve on Held-Out Test Set")
ax.set_xlim(0,1.02); ax.set_ylim(0,1.05); ax.legend()
fig.tight_layout();
fig.savefig(OUT/"17_precision_recall_curve.png",dpi=180);
plt.close(fig)

thresholds = np.linspace(.01,.99,99); precisions=[]; recalls=[];
specificities=[]
for threshold in thresholds:
    th_pred=(probs>=threshold).astype(int)
```

```
    tn,fp,fn,tp=confusion_matrix(y_test,th_pred,labels=[0,1]).ravel()
    precisions.append(tp/(tp+fp) if tp+fp else np.nan)
    recalls.append(tp/(tp+fn) if tp+fn else np.nan)
    specificities.append(tn/(tn+fp) if tn+fp else np.nan)
fig, ax=plt.subplots(figsize=(8,5.3))
ax.plot(thresholds,precisions,label="Precision",color="#2563EB")
ax.plot(thresholds,recalls,label="Recall /
sensitivity",color="#DC2626")
ax.plot(thresholds,specificities,label="Specificity",color="#059669")
ax.axvline(.5,ls="--",color="#6B7280",lw=1,label="Default threshold =
0.50")
ax.set_title("Decision Threshold Trade-offs on Test Cases");
ax.set_xlabel("Probability threshold for malignant")
ax.set_ylabel("Score"); ax.set_ylim(-.02,1.03); ax.legend(loc="best")
fig.tight_layout();
fig.savefig(OUT/"18_threshold_tradeoff.png",dpi=180); plt.close(fig)

prob_df=pd.DataFrame({"Predicted probability of malignant":probs,
                      "Actual
diagnosis":np.where(y_test==1,"malignant","benign")})
fig,ax=plt.subplots(figsize=(7.5,5))
sns.histplot(data=prob_df,x="Predicted probability of
malignant",hue="Actual diagnosis",bins=14,
             stat="count",element="step",common_norm=False,
             palette={"benign":"#10B981","malignant":"#F97316"},ax=ax)
ax.set_title("Predicted Malignancy Probabilities by Actual Class");
ax.set_xlim(0,1)
fig.tight_layout();
fig.savefig(OUT/"19_probability_distribution.png",dpi=180);
plt.close(fig)

sort_idx=np.argsort(probs);
point_colors=np.where(y_test[sort_idx]==1,"#F97316","#10B981")
fig,ax=plt.subplots(figsize=(10,4.8));
ax.scatter(np.arange(len(sort_idx)),probs[sort_idx],c=point_colors,s=3
5,alpha=.85)
ax.axhline(.5,color="#111827",ls="--",label="Decision threshold =
0.50")
ax.set_xlabel("Test cases sorted by predicted malignancy
probability"); ax.set_ylabel("Predicted probability of malignant")
ax.set_title("Sorted Test Predictions (orange = actual malignant;
green = actual benign)")
ax.set_ylim(-.03,1.03); ax.legend()
fig.tight_layout();
fig.savefig(OUT/"20_sorted_test_predictions.png",dpi=180);
plt.close(fig)

# Architecture, split composition, and approximate validation
permutation importance.
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch
```

```
fig,ax=plt.subplots(figsize=(10,4.6)); ax.axis("off")
blocks=[(.03,.34,.16,.34,"Input layer\n30 features","#DBEAFE"),
(.25,.34,.19,.34,"Dense: 64\nReLU\nDropout 0.20","#D1FAE5"),
(.50,.34,.19,.34,"Dense: 32\nReLU\nDropout 0.10","#FEF3C7"),
(.77,.34,.19,.34,"Output: 1 logit\nSigmoid for
probability","#FCE7F3")]
for x0,y0,w,h,label,fc in blocks:

ax.add_patch(FancyBboxPatch((x0,y0),w,h,boxstyle="round,pad=0.012,roun
ding_size=0.02",fc=fc,ec="#374151",lw=1.2))
    ax.text(x0+w/2,y0+h/2,label,ha="center",va="center",fontsize=10)
for i in range(len(blocks)-1):
    ax.add_patch(FancyArrowPatch((blocks[i][0]+blocks[i][2]+.01,.51),
(blocks[i+1][0]-.01,.51),arrowstyle="-|
>",mutation_scale=15,color="#4B5563",lw=1.5))
ax.text(.5,.15,"BCEWithLogitsLoss | Adam, lr=0.001 | weight
decay=0.0001 | validation-loss early
stopping",ha="center",va="center",fontsize=9.5,wrap=True)
ax.set_title("Multilayer Perceptron (MLP)
Architecture",fontsize=15,pad=10)
fig.tight_layout();
fig.savefig(OUT/"21_model_architecture.png",dpi=180,bbox_inches="tight
"); plt.close(fig)

split_counts=pd.DataFrame({

"train":pd.Series(y_train).map({0:"benign",1:"malignant"}).value_count
s(),

"validation":pd.Series(y_val).map({0:"benign",1:"malignant"}).value_co
unts(),

"test":pd.Series(y_test).map({0:"benign",1:"malignant"}).value_counts(
)
}).fillna(0).reindex(["benign","malignant"])
fig,ax=plt.subplots(figsize=(8,5));
split_counts.T.plot(kind="bar",ax=ax,color=["#10B981","#F97316"])
ax.set_title("Class Distribution Preserved Across Data Splits");
ax.set_xlabel("Partition"); ax.set_ylabel("Number of cases")
ax.tick_params(axis="x",rotation=0); ax.legend(title="Diagnosis")
fig.tight_layout();
fig.savefig(OUT/"22_split_distribution.png",dpi=180); plt.close(fig)

with torch.no_grad():

validation_logits=model(torch.tensor(X_val,dtype=torch.float32,device=
device)).view(-1).cpu().numpy()
validation_probs=1/(1+np.exp(-validation_logits));
validation_pred=(validation_probs>=.5).astype(int)
base_val_acc=accuracy_score(y_val,validation_pred);
```

```
rng=np.random.default_rng(SEED); importance_rows=[]
for feat_name in top_diff_features[:10]:
    feat_idx=feature_cols.index(feat_name); drops=[]
    for _ in range(10):
        xperm=X_val.copy();
xperm[:,feat_idx]=rng.permutation(xperm[:,feat_idx])
        with torch.no_grad():

lp=model(torch.tensor(xperm,dtype=torch.float32,device=device)).view(-
1).cpu().numpy()
        pp=1/(1+np.exp(-lp)); drops.append(base_val_acc-
accuracy_score(y_val,(pp>=.5).astype(int)))

importance_rows.append({"feature":feat_name,"mean_accuracy_drop":float
(np.mean(drops)),"std_accuracy_drop":float(np.std(drops))})
importance_df=pd.DataFrame(importance_rows).sort_values("mean_accuracy
_drop",ascending=True)
importance_df.to_csv(OUT/"permutation_importance_validation.csv",index
=False)
fig,ax=plt.subplots(figsize=(9.5,6))
ax.barh(importance_df["feature"],importance_df["mean_accuracy_drop"],x
err=importance_df["std_accuracy_drop"],color="#6366F1",alpha=.9,capsiz
e=2)
ax.axvline(0,color="#111827",lw=.8); ax.set_title("Approximate
Permutation Importance on Validation Set")
ax.set_xlabel("Decrease in validation accuracy after feature
shuffling")
fig.tight_layout();
fig.savefig(OUT/"23_permutation_importance.png",dpi=180);
plt.close(fig)

rep=classification_report(y_test,pred,labels=[0,1],target_names=["beni
gn","malignant"],output_dict=True,zero_division=0)
class_metrics=pd.DataFrame(rep).T.loc[["benign","malignant"],
["precision","recall","f1-score"]]
fig,ax=plt.subplots(figsize=(6.5,3.4));
sns.heatmap(class_metrics,annot=True,fmt=".3f",vmin=0,vmax=1,cmap="YlG
nBu",linewidths=.5,ax=ax)
ax.set_title("Per-Class Test Metrics"); fig.tight_layout();
fig.savefig(OUT/"24_classification_report_heatmap.png",dpi=180);
plt.close(fig)

metric_plot=bench_df[["accuracy","precision_malignant","recall_maligna
nt","f1_malignant","roc_auc"]]
fig,ax=plt.subplots(figsize=(10,5.7));
metric_plot.plot(kind="bar",ax=ax,width=.82)
ax.set_title("Held-Out Test Metrics: MLP vs Baseline Models");
ax.set_ylabel("Score"); ax.set_xlabel("Model")
ax.set_ylim(0,1.05); ax.tick_params(axis="x",rotation=0);
ax.legend(title="Metric",ncol=2,loc="lower right")
```

```
fig.tight_layout();
fig.savefig(OUT/"25_model_comparison.png",dpi=180); plt.close(fig)

frac_pos,mean_pred=calibration_curve(y_test,probs,n_bins=5,strategy="q
uantile")
fig,ax=plt.subplots(figsize=(6.8,5.2));
ax.plot(mean_pred,frac_pos,"o-",label="MLP",color="#7C3AED")
ax.plot([0,1],[0,1],"--",label="Ideal calibration",color="#6B7280")
ax.set_xlabel("Mean predicted probability"); ax.set_ylabel("Observed
malignant fraction")
ax.set_title("Probability Calibration Check (5 Quantile Bins)");
ax.set_xlim(0,1); ax.set_ylim(0,1); ax.legend()
fig.tight_layout();
fig.savefig(OUT/"26_calibration_curve.png",dpi=180); plt.close(fig)

pair_features=["mean radius","mean texture","mean perimeter","mean
concavity"]
pair_grid=sns.pairplot(df[pair_features+
["diagnosis"]],hue="diagnosis",diag_kind="hist",corner=True,

palette={"benign":"#10B981","malignant":"#F97316"},plot_kws={"alpha":.
65,"s":20},height=1.75)
pair_grid.fig.suptitle("Pairwise Relationships Among Selected
Measurements",y=1.02)
pair_grid.fig.savefig(OUT/"27_selected_feature_pairplot.png",dpi=170,b
box_inches="tight"); plt.close(pair_grid.fig)

grid_features=["mean radius","mean texture","mean smoothness","mean
concavity","worst perimeter","worst area"]
fig,axes=plt.subplots(2,3,figsize=(13,7.5))
for ax,feature in zip(axes.ravel(),grid_features):

sns.histplot(data=df,x=feature,hue="diagnosis",kde=True,element="step"
,stat="density",common_norm=False,

palette={"benign":"#10B981","malignant":"#F97316"},ax=ax)
    ax.set_title(feature.title()); ax.set_ylabel("Density")
fig.suptitle("Representative Feature Distributions by
Diagnosis",y=1.01,fontsize=15)
fig.tight_layout();
fig.savefig(OUT/"28_feature_distribution_grid.png",dpi=180,bbox_inches
="tight"); plt.close(fig)

# End-to-end workflow visual, keeping reproducibility decisions
visible.
fig,ax=plt.subplots(figsize=(12,5.2)); ax.set_xlim(0,1);
ax.set_ylim(0,1); ax.axis("off")
workflow_blocks=[(.025,.55,.13,.25,"1. Data\nloading","#DBEAFE"),
(.19,.55,.14,.25,"2. Quality\nchecks","#E0E7FF"),
(.365,.55,.14,.25,"3. EDA +\nvisualization","#D1FAE5"),
```

```
(.54,.55,.14,.25,"4. Stratified\nsplitting","#FEF3C7"),
(.715,.55,.12,.25,"5. Scale +\ntrain MLP","#FCE7F3"),
(.87,.55,.105,.25,"6. Evaluate\n+ compare","#FFEDD5")]
for x0,y0,w,h,label,fc in workflow_blocks:

ax.add_patch(FancyBboxPatch((x0,y0),w,h,boxstyle="round,pad=0.012,roun
ding_size=0.02",fc=fc,ec="#334155",lw=1.2))

ax.text(x0+w/2,y0+h/2,label,ha="center",va="center",fontsize=10,fontwe
ight="bold")
for i in range(len(workflow_blocks)-1):
    ax.add_patch(FancyArrowPatch((workflow_blocks[i]
[0]+workflow_blocks[i][2]+.005,.675),(workflow_blocks[i+1]
[0]-.005,.675),arrowstyle="-|
>",mutation_scale=15,color="#475569",lw=1.4))
ax.add_patch(FancyBboxPatch((.18,.12),.64,.20,boxstyle="round,pad=0.01
5,rounding_size=0.02",fc="#F8FAFC",ec="#64748B",lw=1))
ax.text(.5,.22,"Fixed seed | training-only scaler | validation early
stopping | held-out
test",ha="center",va="center",fontsize=10,wrap=True)
ax.add_patch(FancyArrowPatch((.77,.54),(.77,.34),arrowstyle="-|
>",mutation_scale=14,color="#64748B",lw=1.2))
ax.set_title("End-to-End Analysis Workflow",fontsize=16,pad=12)
fig.tight_layout();
fig.savefig(OUT/"29_end_to_end_workflow.png",dpi=180,bbox_inches="tigh
t"); plt.close(fig)

metrics["baseline_models"]={name:{key:float(value) for key,value in
row.items()} for name,row in benchmarks.items()}
metrics["average_precision_malignant"]=float(avg_precision)
metrics["output_figure_count"]=len(list(OUT.glob("*.png")))
(OUT/"metrics.json").write_text(json.dumps(metrics,indent=2))
print("\nBaseline comparison:")
print(bench_df.round(4).to_string())
print(f"\nExpanded output figure count:
{metrics['output_figure_count']}")
```